

```
In [1]: import pandas as pd
import sqlite3
import seaborn as sns
import matplotlib.pyplot as plt
```

```
In [2]: conn = sqlite3.connect(r"D:\data science\inventory invoice intellignece ml\data\inv
print("Connected successfully")
```

Connected successfully

```
In [4]: tables = pd.read_sql_query(
    "SELECT name FROM sqlite_master WHERE type='table';",
    conn
)

print(tables)
```

```
      name
0  purchases
1  purchase_prices
2  vendor_invoice
3  begin_inventory
4  end_inventory
```

```
In [5]: for table in tables['name']:
    print('Table name', table)
    df = pd.read_sql_query(f"SELECT * FROM {table} LIMIT 5", conn)
    display(df)
```

Table name purchases

	InventoryId	Store	Brand	Description	Size	VendorNumber	VendorName
0	69_MOUNTMEND_8412	69	8412	Tequila Ocho Plata Fresno	750mL	105	ALTAMAR BRANDS LLC
1	30_CULCHETH_5255	30	5255	TGI Fridays Ultimte Mudslide	1.75L	4466	AMERICAN VINTAGE BEVERAGE
2	34_PITMERDEN_5215	34	5215	TGI Fridays Long Island Iced	1.75L	4466	AMERICAN VINTAGE BEVERAGE
3	1_HARDERSFIELD_5255	1	5255	TGI Fridays Ultimte Mudslide	1.75L	4466	AMERICAN VINTAGE BEVERAGE
4	76_DONCASTER_2034	76	2034	Glendalough Double Barrel	750mL	388	ATLANTIC IMPORTING COMPANY

Table name purchase_prices

	Brand	Description	Price	Size	Volume	Classification	PurchasePrice	VendorNumber
0	58	Gekkeikan Black & Gold Sake	12.99	750mL	750	1	9.28	8320
1	62	Herradura Silver Tequila	36.99	750mL	750	1	28.67	1128
2	63	Herradura Reposado Tequila	38.99	750mL	750	1	30.46	1128
3	72	No. 3 London Dry Gin	34.99	750mL	750	1	26.11	9165
4	75	Three Olives Tomato Vodka	14.99	750mL	750	1	10.94	7245



Table name vendor_invoice

	VendorNumber	VendorName	InvoiceDate	PONumber	PODate	PayDate	Quantity	D
0	105	ALTAMAR BRANDS LLC	2024-01-04	8124	2023- 12-21	2024- 02-16	6	2
1	4466	AMERICAN VINTAGE BEVERAGE	2024-01-07	8137	2023- 12-22	2024- 02-21	15	1
2	388	ATLANTIC IMPORTING COMPANY	2024-01-09	8169	2023- 12-24	2024- 02-16	5	1
3	480	BACARDI USA INC	2024-01-12	8106	2023- 12-20	2024- 02-05	10100	1374
4	516	BANFI PRODUCTS CORP	2024-01-07	8170	2023- 12-24	2024- 02-12	1935	155



Table name begin_inventory

	InventoryId	Store	City	Brand	Description	Size	onHand	Price	sta
0	1_HARDERSFIELD_58	1	HARDERSFIELD	58	Gekkeikan Black & Gold Sake	750mL	8	12.99	20
1	1_HARDERSFIELD_60	1	HARDERSFIELD	60	Canadian Club 1858 VAP	750mL	7	10.99	20
2	1_HARDERSFIELD_62	1	HARDERSFIELD	62	Herradura Silver Tequila	750mL	6	36.99	20
3	1_HARDERSFIELD_63	1	HARDERSFIELD	63	Herradura Reposado Tequila	750mL	3	38.99	20
4	1_HARDERSFIELD_72	1	HARDERSFIELD	72	No. 3 London Dry Gin	750mL	6	34.99	20



Table name end_inventory

	InventoryId	Store	City	Brand	Description	Size	onHand	Price	en
0	1_HARDERSFIELD_58	1	HARDERSFIELD	58	Gekkeikan Black & Gold Sake	750mL	11	12.99	
1	1_HARDERSFIELD_62	1	HARDERSFIELD	62	Herradura Silver Tequila	750mL	7	36.99	
2	1_HARDERSFIELD_63	1	HARDERSFIELD	63	Herradura Reposado Tequila	750mL	7	38.99	
3	1_HARDERSFIELD_72	1	HARDERSFIELD	72	No. 3 London Dry Gin	750mL	4	34.99	
4	1_HARDERSFIELD_75	1	HARDERSFIELD	75	Three Olives Tomato Vodka	750mL	7	14.99	



```
In [7]: purchase_agg_df=pd.read_sql_query("""
select
p.PONumber,
count(distinct p.Brand) as total_brands,
sum(p.Dollars) as total_item_dollars,
sum(p.Quantity) as total_item_quantity,
avg( julianday(p.ReceivingDate)-julianday(p.PODate)) as avg_receiving_delay
from purchases p
```

```
group by p.PONumber
""",conn)
```

```
In [9]: pd.read_sql_query("""
        SELECT
            vi.Quantity AS invoice_quantity,
            vi.Dollars AS invoice_Dollars,
            vi.Freight,
            (julianday(vi.InvoiceDate) - julianday(vi.PODate)) AS days_po_to_invoice,
            (julianday(vi.PayDate) - julianday(vi.InvoiceDate)) AS days_to_pay
        FROM vendor_invoice vi
        """, conn)
```

```
Out[9]:
```

	invoice_quantity	invoice_Dollars	Freight	days_po_to_invoice	days_to_pay
0	6	214.26	3.47	14.0	43.0
1	15	140.55	8.57	16.0	45.0
2	5	106.60	4.61	16.0	38.0
3	10100	137483.78	2935.20	23.0	24.0
4	1935	15527.25	429.20	14.0	36.0
...	...	...	...	...	...
5538	90	1563.00	8.60	16.0	35.0
5539	4617	37300.48	186.50	18.0	39.0
5540	9848	202815.78	932.95	11.0	33.0
5541	24747	149007.56	819.54	14.0	36.0
5542	437	3608.11	16.60	18.0	38.0

5543 rows × 5 columns

```
In [13]: df=pd.read_sql_query("""
        WITH purchase_agg AS (
            SELECT
                p.PONumber,
                COUNT(DISTINCT p.Brand) AS total_brands,
                SUM(p.Quantity) AS total_item_quantity,
                SUM(p.Dollars) AS total_item_dollars,
                AVG(julianday(p.ReceivingDate) - julianday(p.PODate)) AS avg_receiving_dela
            FROM purchases p
            GROUP BY p.PONumber
        )

        SELECT
            vi.PONumber,
            vi.Quantity AS invoice_quantity,
            vi.Dollars AS invoice_dollars,
            vi.Freight,
            (julianday(vi.InvoiceDate) - julianday(vi.PODate)) AS days_po_to_invoice,
```

```

        (julianday(vi.PayDate) - julianday(vi.InvoiceDate)) AS days_to_pay,
        pa.total_brands,
        pa.total_item_quantity,
        pa.total_item_dollars,
        pa.avg_receiving_delay
    FROM vendor_invoice vi
    LEFT JOIN purchase_agg pa
        ON vi.PONumber = pa.PONumber
    """ , conn)

```

In [14]: `df.isnull().sum()`

```

Out[14]: PONumber          0
         invoice_quantity  0
         invoice_dollars   0
         Freight          0
         days_po_to_invoice 0
         days_to_pay       0
         total_brands      0
         total_item_quantity 0
         total_item_dollars 0
         avg_receiving_delay 0
         dtype: int64

```

In [15]: `df.dtypes`

```

Out[15]: PONumber          int64
         invoice_quantity  int64
         invoice_dollars   float64
         Freight          float64
         days_po_to_invoice float64
         days_to_pay       float64
         total_brands      int64
         total_item_quantity int64
         total_item_dollars float64
         avg_receiving_delay float64
         dtype: object

```

```

In [19]: def create_invoice_risk_label(row):
         if(abs(row["invoice_dollars"]-row["total_item_dollars"])>5):
             return 1
         if row["avg_receiving_delay"]>10:
             return 1
         return 0
         df["flag_invoice"]=df.apply(create_invoice_risk_label,axis=1)
         df["flag_invoice"].value_counts()

```

```

Out[19]: flag_invoice
         0    3693
         1    1850
         Name: count, dtype: int64

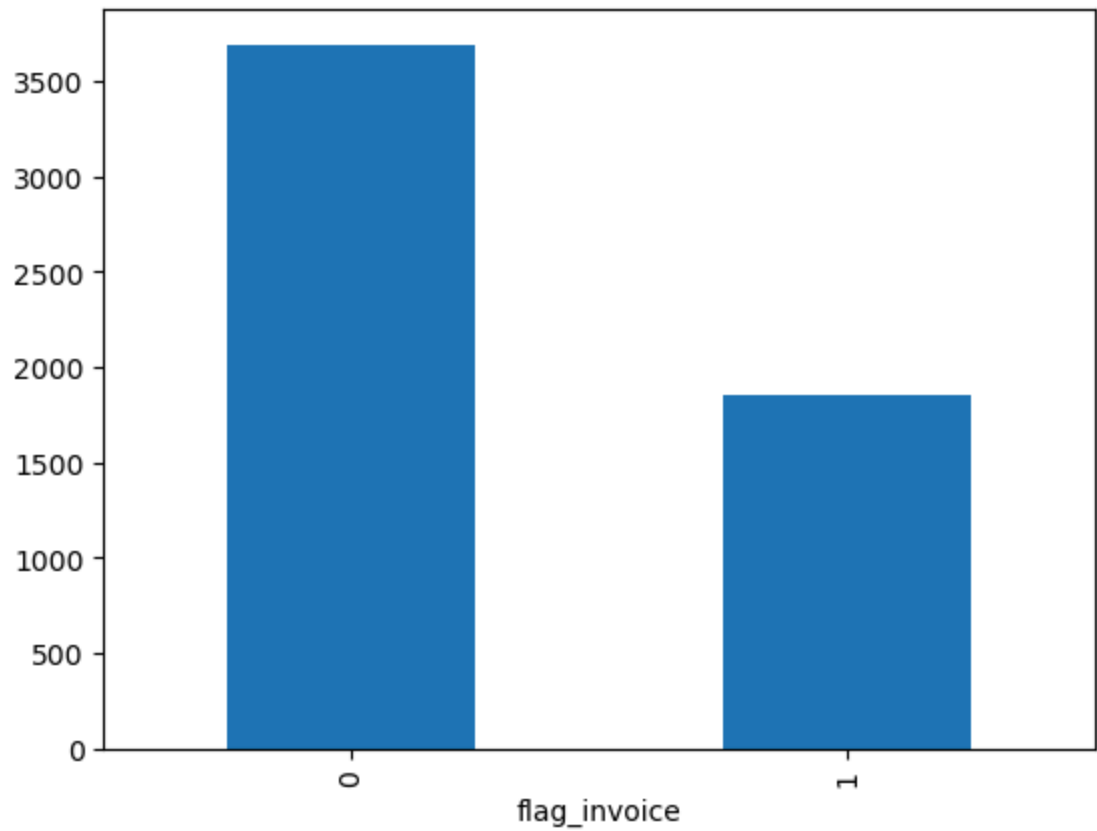
```

In [20]: `df['flag_invoice'].value_counts().plot(kind='bar')`

```

Out[20]: <Axes: xlabel='flag_invoice'>

```



In [21]:

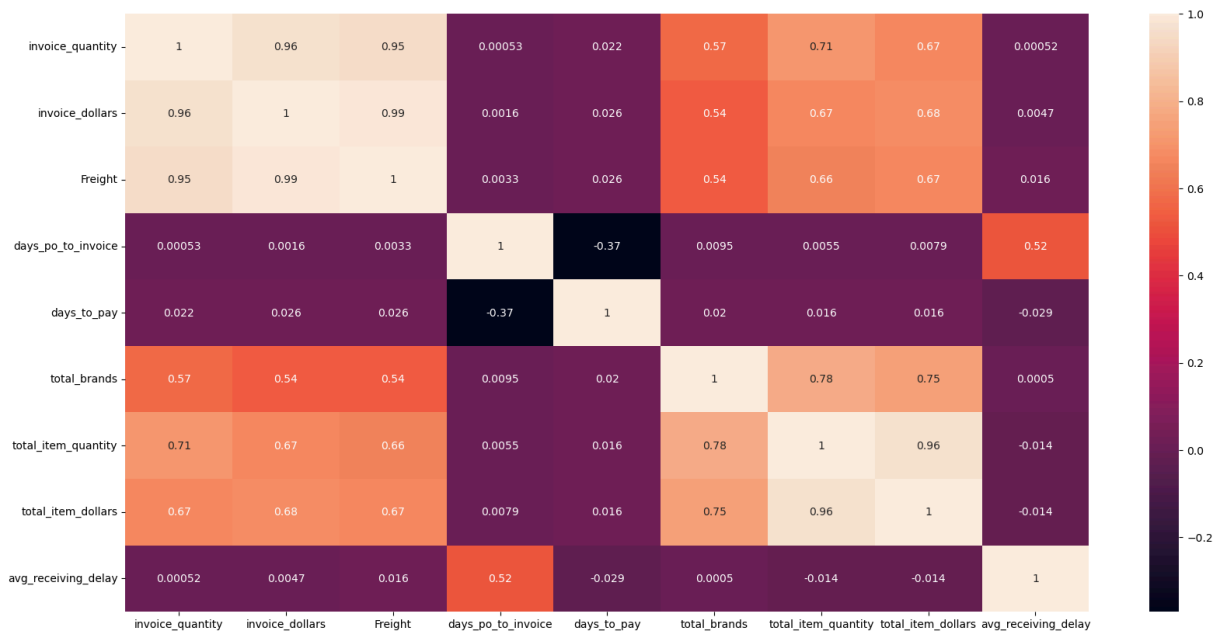
df.corr()

Out[21]:

	PONumber	invoice_quantity	invoice_dollars	Freight	days_po_to_inv
PONumber	1.000000	0.049230	0.058917	0.043617	-0.002
invoice_quantity	0.049230	1.000000	0.963831	0.946550	0.000
invoice_dollars	0.058917	0.963831	1.000000	0.985141	0.001
Freight	0.043617	0.946550	0.985141	1.000000	0.003
days_po_to_invoice	-0.002234	0.000526	0.001610	0.003265	1.000
days_to_pay	0.012558	0.022284	0.025661	0.025513	-0.370
total_brands	0.013423	0.566394	0.536952	0.536381	0.009
total_item_quantity	0.049442	0.706117	0.667666	0.656616	0.005
total_item_dollars	0.059221	0.668171	0.679537	0.670768	0.007
avg_receiving_delay	-0.042362	0.000519	0.004694	0.015645	0.515
flag_invoice	0.564946	0.032783	0.037994	0.037759	0.183

In [24]:

plt.figure(figsize=(20,10))
sns.heatmap(df.iloc[:,1:-1].corr(),annot=True)
plt.show()



```
In [25]: flagged=df[df['flag_invoice']==1]
normal=df[df['flag_invoice']==0]
```

```
In [26]: significant_features=[]
non_significant_features=[]
results=[]
```

```
In [27]: metrics=['invoice_quantity','invoice_dollars','Freight','days_po_to_invoice','days_
```

```
In [28]: from scipy.stats import ttest_ind
for metric in metrics:
    flagged_mean=flagged[metric].mean()
    normal_mean=normal[metric].mean()
    t_stat,p_value=ttest_ind(
        flagged[metric].dropna(),
        normal[metric].dropna(),
        equal_var=False
    )
    if p_value <0.05:
        significant_features.append(metric)
        results.append({
            "metric":metric,
            "flagged_mean":flagged_mean.round(2),
            "normal_mean":normal_mean.round(2),
            "p_value":p_value.round(3)
        })
    else:
        non_significant_features.append(metric)
```

```
In [29]: non_significant_features
```

```
Out[29]: ['days_to_pay', 'total_brands']
```

```
In [30]: significant_features
```

```
Out[30]: ['invoice_quantity',  
         'invoice_dollars',  
         'Freight',  
         'days_po_to_invoice',  
         'total_item_quantity',  
         'total_item_dollars',  
         'avg_receiving_delay']
```

```
In [31]: results
```

```
Out[31]: [{'metric': 'invoice_quantity',  
          'flagged_mean': 6728.28,  
          'normal_mean': 5723.55,  
          'p_value': 0.021},  
         {'metric': 'invoice_dollars',  
          'flagged_mean': 65600.61,  
          'normal_mean': 54302.64,  
          'p_value': 0.008},  
         {'metric': 'Freight',  
          'flagged_mean': 334.02,  
          'normal_mean': 276.89,  
          'p_value': 0.008},  
         {'metric': 'days_po_to_invoice',  
          'flagged_mean': 17.23,  
          'normal_mean': 16.02,  
          'p_value': 0.0},  
         {'metric': 'total_item_quantity',  
          'flagged_mean': 6728.28,  
          'normal_mean': 5723.55,  
          'p_value': 0.021},  
         {'metric': 'total_item_dollars',  
          'flagged_mean': 65600.61,  
          'normal_mean': 54302.64,  
          'p_value': 0.008},  
         {'metric': 'avg_receiving_delay',  
          'flagged_mean': 8.47,  
          'normal_mean': 7.27,  
          'p_value': 0.0}]
```

```
In [32]: X=df[['invoice_quantity', 'invoice_dollars', 'Freight', 'days_po_to_invoice', 'total_it  
y=df['flag_invoice']
```

```
In [33]: X.describe().round()
```

Out[33]:

	invoice_quantity	invoice_dollars	Freight	days_po_to_invoice	total_item_quantity	tc
count	5543.0	5543.0	5543.0	5543.0	5543.0	
mean	6059.0	58073.0	296.0	16.0	6059.0	
std	14453.0	140234.0	714.0	3.0	14453.0	
min	1.0	4.0	0.0	9.0	1.0	
25%	83.0	968.0	5.0	14.0	83.0	
50%	423.0	4765.0	25.0	16.0	423.0	
75%	5100.0	44587.0	230.0	19.0	5100.0	
max	141660.0	1660436.0	8468.0	23.0	141660.0	

In [37]: `from sklearn.model_selection import train_test_split`

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

In [38]: `from sklearn.preprocessing import StandardScaler, MinMaxScaler`

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

In [39]: `# Imports`

```
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
```

In [40]: `# Models`

```
model1 = LogisticRegression()
model2 = DecisionTreeClassifier()
model3 = RandomForestClassifier()

# Fit models
model1.fit(X_train_scaled, y_train) # Logistic Regression (use scaled data)
model2.fit(X_train, y_train)       # Decision Tree
model3.fit(X_train, y_train)       # Random Forest
```

Out[40]: `RandomForestClassifier`

```
RandomForestClassifier()
```

In [47]: `from sklearn.metrics import classification_report`

```
def evaluate_model(model, X_test, y_test, name):
    y_pred = model.predict(X_test)
```

```
print(f"----- {name} -----")
print(classification_report(y_test, y_pred))
print("\n")
```

```
In [49]: evaluate_model(model1, X_test_scaled, y_test, 'Logistic Regression') # Logistic Reg
evaluate_model(model2, X_test, y_test, 'Decision Tree Classifier')           # Decisio
evaluate_model(model3, X_test, y_test, 'Random Forest Classifier')           # Random
```

```
----- Logistic Regression -----
              precision    recall  f1-score   support

     0       0.74       0.93       0.82       725
     1       0.74       0.38       0.50       384

 accuracy          0.74          1109
 macro avg       0.74       0.66       0.66       1109
 weighted avg    0.74       0.74       0.71       1109
```

```
----- Decision Tree Classifier -----
              precision    recall  f1-score   support

     0       0.95       0.97       0.96       725
     1       0.94       0.90       0.92       384

 accuracy          0.94          1109
 macro avg       0.94       0.94       0.94       1109
 weighted avg    0.94       0.94       0.94       1109
```

```
----- Random Forest Classifier -----
              precision    recall  f1-score   support

     0       0.94       1.00       0.97       725
     1       1.00       0.88       0.93       384

 accuracy          0.96          1109
 macro avg       0.97       0.94       0.95       1109
 weighted avg    0.96       0.96       0.96       1109
```

```
In [50]: model3.feature_importances_
```

```
Out[50]: array([0.10299778, 0.10369885, 0.09556703, 0.02807125, 0.16311784,
                0.20807262, 0.29847463])
```

```
In [52]: feature_importance = pd.DataFrame({
        'feature': X_train.columns,
        'importance': model3.feature_importances_
    }).sort_values(by="importance", ascending=False)
```

```
feature_importance
```

Out[52]:

	feature	importance
6	avg_receiving_delay	0.298475
5	total_item_dollars	0.208073
4	total_item_quantity	0.163118
1	invoice_dollars	0.103699
0	invoice_quantity	0.102998
2	Freight	0.095567
3	days_po_to_invoice	0.028071

In [53]:

```
X=df[['invoice_quantity','invoice_dollars','Freight','total_item_quantity','total_i
y=df['flag_invoice']
```

In [54]:

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

In [55]:

```
model3 = RandomForestClassifier()
model3.fit(X_train, y_train)
```

Out[55]:

RandomForestClassifier ⓘ ?

RandomForestClassifier()

In [56]:

```
evaluate_model(model3, X_test, y_test,'Random Forest Classifier')
```

```
----- Random Forest Classifier -----
```

	precision	recall	f1-score	support
0	0.86	0.98	0.92	725
1	0.95	0.71	0.81	384
accuracy			0.88	1109
macro avg	0.91	0.84	0.86	1109
weighted avg	0.89	0.88	0.88	1109

In [57]:

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import make_scorer, f1_score
from sklearn.model_selection import GridSearchCV
```

```

# Initialize the model
rf = RandomForestClassifier(
    random_state=42,
    n_jobs=-1
)

# Define the hyperparameter grid for tuning
param_grid = {
    "n_estimators": [100, 200, 300],
    "max_depth": [None, 4, 5, 6],
    "min_samples_split": [2, 3, 5],
    "min_samples_leaf": [1, 2, 5],
    "criterion": ["gini", "entropy"]
}

# Define the evaluation metric
scorer = make_scorer(f1_score)

# Setup the Grid Search with 5-fold cross-validation
grid_search = GridSearchCV(
    estimator=rf,
    param_grid=param_grid,
    scoring=scorer,
    cv=5,
    verbose=2,
    n_jobs=-1
)

# To run the search, you would call:
grid_search.fit(X_train_scaled, y_train)
evaluate_model(grid_search, X_test_scaled, y_test, 'Random Forest Classifier')

```

Fitting 5 folds for each of 216 candidates, totalling 1080 fits

C:\Users\hp\anaconda3\Lib\site-packages\numpy\ma\core.py:2820: RuntimeWarning: invalid value encountered in cast

```

_data = np.array(data, dtype=dtype, copy=copy,
----- Random Forest Classifier -----
              precision    recall  f1-score   support

         0       0.87        0.98        0.92        725
         1       0.96        0.71        0.82        384

 accuracy          0.89          0.89          0.89        1109
 macro avg         0.91          0.85          0.87        1109
 weighted avg      0.90          0.89          0.89        1109

```

In [58]: `from sklearn.metrics import confusion_matrix`

In [60]: `confusion_matrix(y_test, grid_search.predict(X_test_scaled))`

Out[60]: `array([[713, 12],
 [110, 274]], dtype=int64)`

```
In [63]: confusion_matrix(y_test, model3.predict(X_test_scaled))
```

```
C:\Users\hp\anaconda3\Lib\site-packages\sklearn\base.py:493: UserWarning: X does not have valid feature names, but RandomForestClassifier was fitted with feature names
warnings.warn(
```

```
Out[63]: array([[723,  2],
               [384,  0]], dtype=int64)
```

```
In [64]: grid_search.best_params_
```

```
Out[64]: {'criterion': 'gini',
          'max_depth': None,
          'min_samples_leaf': 1,
          'min_samples_split': 5,
          'n_estimators': 300}
```

```
In [ ]:
```